

GETTING STARTED WITH LLAMA AND OTHER LOCAL MODELS

O'Reilly Live Training — Quick Reference Cheatsheet

Gemma 4 · Ollama · RAG · Tool Calling · Agents · Fine-Tuning · 2026 Edition · Updated October 2026

SETUP & ENVIRONMENT

Environment (conda)

```
conda create -n oreilly-llama3
python=3.10
conda activate oreilly-llama3
pip install -r requirements/
requirements.txt
cp .env.example .env # key optional

# Jupyter
pip install jupyter
python3 -m ipykernel install \
  --user --name=oreilly-llama3
jupyter lab
```

Install Ollama & Pull Models

```
# Install: https://ollama.com/download
ollama pull gemma4          # = e4b 9.6GB
ollama pull gemma4:26b     # 24 GB GPU
ollama pull gemma4:12b     # 16 GB laptop
ollama pull gemma4:e4b     # CPU-only
ollama serve                # :11434
ollama list                 # installed
ollama run gemma4          # chat
ollama rm modelname        # remove
```

1.0 — QUICKSTART WITH OLLAMA

Python API

```
import ollama

# Simple chat
response = ollama.chat(
    model="gemma4",
    messages=[{
        "role": "user",
        "content": "Explain RAG in 2
sentences."
    }]
)
print(response['message']['content'])

# Streaming
stream = ollama.chat(
    model="gemma4",
    messages=[{"role": "user",
"content": "Hi"}],
    stream=True
)
for chunk in stream:
    print(chunk['message']['content'],
end='')
```

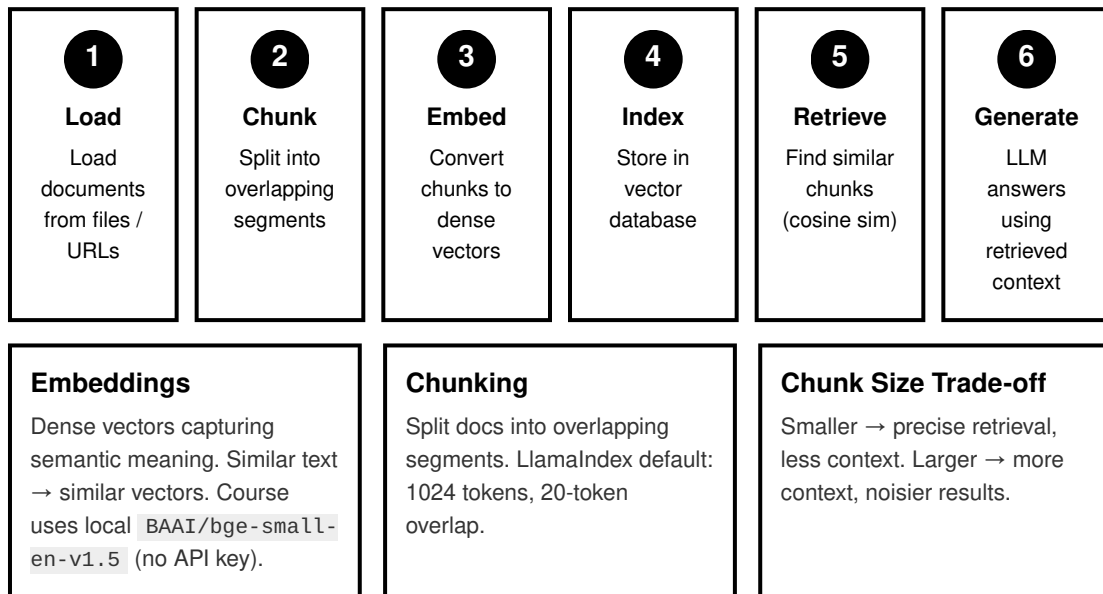
Reusable Helper

```
import ollama

def chat(prompt, model="gemma4",
        system=None):
    messages = []
    if system:
        messages.append({
            "role": "system",
            "content": system
        })
    messages.append({
        "role": "user",
        "content": prompt
    })
    resp = ollama.chat(
        model=model,
        messages=messages
    )
    return resp['message']['content']

# Usage
answer = chat("What is LoRA?")
print(answer)
```

2.0 — INTRODUCTION TO RAG



2.1 — LOCAL RAG WITH GEMMA 4 + LLAMAINDEX

Build Index & Query

```
from llama_index.llms.ollama import Ollama
from llama_index.embeddings.huggingface \
    import HuggingFaceEmbedding
from llama_index.core import (
    VectorStoreIndex,
    SimpleDirectoryReader, Settings
)

Settings.llm = Ollama(
    model="gemma4",
    request_timeout=120.0
)
Settings.embed_model = HuggingFaceEmbedding(
    model_name="BAAI/bge-small-en-v1.5"
)
docs = SimpleDirectoryReader("data/").load_data()
index = VectorStoreIndex.from_documents(docs)
engine = index.as_query_engine(
    similarity_top_k=3
)
response = engine.query("What is LoRA?")
```

Inspect Sources & Persist

```
# View retrieved source chunks
for node in response.source_nodes:
    print(node.text[:200])
    print(node.metadata)

# Persist index to disk
index.storage_context.persist(
    persist_dir="./storage"
)

# Reload from disk
from llama_index.core import (
    StorageContext,
    load_index_from_storage
)
ctx = StorageContext.from_defaults(
    persist_dir="./storage"
)
index = load_index_from_storage(ctx)
```

RAG Stack		
Component	Tool	Why
LLM	ollama (gemma4)	Local, no API key, easy model management
Embeddings	BAAI/bge-small-en-v1.5	384-dim, fast, runs locally (HuggingFace)
Orchestration	LlamaIndex	Handles chunking, indexing, retrieval
Vector Store	In-memory / ChromaDB / FAISS	In-memory for dev, ChromaDB or FAISS for production

3.0 — TOOL CALLING WITH OLLAMA

Define & Register Tools	Execute Tool Calls
<pre> import ollama def get_weather(city: str) -> str: return f"Weather in {city}: 22°C sunny" tools = [{ "type": "function", "function": { "name": "get_weather", "description": "Get weather for a city", "parameters": { "type": "object", "properties": { "city": {"type": "string"} }, "required": ["city"] } } }] response = ollama.chat(model="gemma4", messages=[{ "role": "user", "content": "Weather in Paris?" }], tools=tools) </pre>	<pre> tools_map = {"get_weather": get_weather} if response.message.tool_calls: for tc in response.message.tool_calls: fn = tools_map[tc.function.name] result = fn(**tc.function.arguments) # Feed result back to model messages = [response.message, { "role": "tool", "content": str(result), "name": tc.function.name }] final = ollama.chat(model="gemma4", messages=messages) print(final.message.content) </pre>

3.1 — LOCAL AGENTS: STRUCTURED OUTPUTS → TOOL CALLING

Structured Outputs (Pydantic)

```
from pydantic import BaseModel
import ollama

class Task(BaseModel):
    title: str
    priority: str # high/medium/low
    estimated_hours: int

response = ollama.chat(
    model="gemma4",
    messages=[{
        "role": "user",
        "content": "Create a task to
build login page"
    }],
    format=Task.model_json_schema()
)
task = Task.model_validate_json(
    response.message.content
)
print(task.title, task.priority)
```

Agent Concepts Summary

```
# Agent Loop pattern:
# User → LLM decides tool → Execute
tool
# → Feed result back → LLM final
response

# Concept      Ollama Feature
# -----
# -----
# Structured
format=Schema.model_json_schema()
# Outputs
#
# Tool Calling  tools=[{type,
function...}]
#              (LLM decides which to
call)
#
# Agent Loop    Combine both:
#              1. Send messages +
tools
#              2. Execute tool_calls
#              3. Append tool results
#              4. Re-call chat()
until
#              no tool_calls
remain
```

4.0 — SIMPLE AGENTIC RAG: REACT PATTERN

Simple ReAct Agent (Reason → Act → Observe loop)

```
import ollama

class SimpleAgent:
    def __init__(self, model="gemma4", tools=None, tools_map=None, max_turns=10):
        self.model, self.tools = model, tools or []
        self.tools_map, self.max_turns = tools_map or {}, max_turns
        self.messages = []

    def run(self, user_input: str) -> str:
        self.messages.append({"role": "user", "content": user_input})
        for turn in range(self.max_turns):
            response = ollama.chat(model=self.model, messages=self.messages,
tools=self.tools)
            if not response.message.tool_calls:
                return response.message.content # No tool call = final
            answer
            self.messages.append(response.message)
            for tc in response.message.tool_calls: # Execute each tool
                result = self.tools_map[tc.function.name]**tc.function.arguments)
                self.messages.append({"role": "tool", "content": str(result),
"name": tc.function.name})
            return "Max turns reached."

# Extend by: adding tools, adjusting max_turns, custom system prompt, adding
memory
```

ReAct Loop: Reason → Act → Observe (repeat until done)

- **Turn 1:** Agent receives query, calls tool (e.g. `find_pdf_files`) to discover PDFs, then `search_pdf`
- **Turn N:** Agent receives tool results, may call more tools or synthesise final answer
- **Done:** When LLM returns a response with no `tool_calls`

5.0 — USEFUL LOCAL AGENTS: WHEN LOCAL VS CLOUD

Local Agent ✓ Use When...

- Privacy matters — data must stay on device
- Cost control — no per-token billing
- Single-tool chains with deterministic output
- Private document analysis
- Offline / air-gapped environments
- Hermes Agent: `ollama launch hermes`
— agents need ≥64K ctx:
`OLLAMA_CONTEXT_LENGTH=64000 ollama serve`

Cloud Agent ✓ Use When...

- Long-context synthesis (>64K tokens)
- Multi-hop reasoning chains (complex logic)
- Code edits beyond ~200 LOC
- Ambiguous planning needing world knowledge
- High-stakes decisions (medical, legal, financial, irreversible)

6.0 / 6.1 — FINE-TUNING GEMMA 4 (LORA, QLoRA, PEFT)

Methods

Method	Description
SFT	Supervised Fine-Tuning: instruction→response pairs
RLHF	Reinforcement Learning from Human Feedback
PEFT	Update only a small subset of parameters
LoRA	Train small adapter matrices A,B (10,000× fewer params)
QLoRA	4-bit quantized base + LoRA adapters in higher precision

When to Fine-Tune

- **Cost:** Smaller fine-tuned model can match a larger base model
- **Latency:** Smaller model = faster responses
- **Consistency:** Model reliably follows specific format/style

Start with prompting. Fine-tune only when product shape is finalized and use case is clear.

Hardware (QLoRA min.):

- GPU: 24GB VRAM (RTX 3090/4090)
- Course demo: `unsloth/gemma-4-E2B-it` QLoRA fits ~8GB VRAM
- RAM: 32GB+ system memory
- Storage: 100GB+ SSD free

Tools:

Unsloth · TRL (SFTTrainer) · Axolotl · LLaMA-Factory · AutoTrain · Together AI

6.2 — QUANTIZATION

Precision Levels

Precision	Relative Size	Primary Use	Notes
FP32	Baseline (1×)	Training	Full precision, maximum memory
FP16 / BF16	2× smaller	Fine-tuning	Default for most training pipelines
INT8	4× smaller	Inference	Minimal quality loss
INT4 (GGUF/ AWQ)	8× smaller	Consumer GPU / CPU	2× smaller than INT8; QLoRA uses NF4

Q4 Key Insight

INT4 (Q4) cuts weight memory **8× vs FP32** (4× vs FP16): a 7B model drops from ~26 GB (FP32) to ~3.3 GB.

MoE caveat: Gemma 4 26B activates only 3.8B params per token (fast), but all weights still load — **~18 GB VRAM at Q4** (19 GB download).

8.0 — BEST LOCAL MODELS 2026

Hardware Tiers — Recommended Models

Tier	Hardware	Top Pick	Ollama Tag	Strength
1	CPU, 8–16GB RAM	Qwen3 4B ★	<code>qwen3:4b</code>	General, multilingual, 2.5GB
1	CPU, 8–16GB RAM	Gemma 4 E4B ★ (default)	<code>gemma4</code>	= <code>gemma4:e4b</code> , 9.6GB; image (audio via HF only)
1	CPU, reasoning	Phi-4 Mini	<code>phi4-mini</code>	Reasoning/math at 3.8B (MIT)
2	16GB laptop / unified	Gemma 4 12B ★	<code>gemma4:12b</code>	Image; audio via HF Transformers, not Ollama
2	10–16GB VRAM	Phi-4 14B ★	<code>phi4</code>	Reasoning, STEM, 9.1GB (MIT)
2	8GB VRAM	Qwen3 8B	<code>qwen3:8b</code>	Coding, multilingual, thinking mode
3	24GB VRAM	Qwen 3.8 27B ★	<code>qwen3.8:27b</code>	Coding + agents, 18GB, Aug 2026
3	~18GB VRAM	Gemma 4 26B	<code>gemma4:26b</code>	MoE, 3.8B active, 19GB; no audio
3	24GB VRAM	Gemma 4 31B	<code>gemma4:31b</code>	Flagship Gemma 4, 20GB
3	20GB VRAM	DeepSeek-R1 32B	<code>deepseek-r1:32b</code>	Math + chain-of-thought (MIT)
4	48GB+ VRAM	DeepSeek-R1 70B	<code>deepseek-r1:70b</code>	43GB; MIT; single A100 80GB

Tier 1 — Quick Start Commands (CPU)

```
ollama pull gemma4      # default
ollama pull qwen3:4b    # general
ollama pull phi4-mini   # reasoning
ollama pull gemma4:e2b  # ≤8GB RAM
```

Tier 2–3 — GPU Commands

```
ollama pull gemma4:12b # 16GB RAM
ollama pull gemma4:26b # 24GB GPU
ollama pull qwen3:8b   # 8GB VRAM
ollama pull qwen3.8:27b # agents
ollama pull deepseek-r1:32b
# production serving (vLLM):
vllm serve google/gemma-4-E4B-it
```

COURSE KEY TAKEAWAYS

- **Ollama** — One command to run any open-weight model locally (`ollama pull + ollama.chat()`)
- **RAG** — Load → Chunk → Embed → Index → Retrieve → Generate: give LLMs access to your own documents
- **Tool Calling** — Pass `tools=[...]` to Ollama; LLM decides which function to call and with what args
- **Agents** — Structured outputs (Pydantic) + tool calling + a loop = a capable local agent
- **ReAct** — Reason → Act → Observe loop; keep looping until no `tool_calls` in response
- **Local vs Cloud** — Local wins on privacy, cost and bounded tasks; cloud wins on long context (>64K), multi-hop reasoning and high-stakes decisions
- **Fine-Tuning** — Use QLoRA (4-bit base + LoRA adapters) with Unsloth or TRL on consumer hardware (course: `unsloth/gemma-4-E2B-it`); only fine-tune when task is well-defined
- **Quantization** — INT4 (Q4) weights are 8× smaller than FP32 (4× vs FP16) at minimal quality cost
- **Gemma 4** — Course default `gemma4` (E4B, CPU-friendly); `gemma4:12b` for 16 GB laptops, 26B/31B for 24 GB GPUs; image input on all sizes, Apache 2.0